Tensor Extension Algorithms in Python

Contents provided

The Library mocaxextendpy is a software package written in Python and C that implements some of the Tensor Extension Algorithms presented in the following papers

Steinlechner, M. Riemannian optimization for high-dimensional tensor completion. SIAM J. Sci. Comput. 38. (2016). S461–S484.

Glau, K. Kressner, F. Statti, F. Low-rank tensor approximation for Chebyshev interpolation in parametric option pricing. (2019) https://arxiv.org/abs/1902.04367

The folder we provide has the following content. The root directly contains two folders. One is called Linux and contains all that is related to the software version for Linux. The other is called Windows, which has the corresponding content for the Windows version. There is also a file with the Licence information for this software.

Inside of Linux (Windows) there are three folders: Documentation, Examples, Library.

Documentation contains the html documentation. To read the documentation in this format go to Documentation/html and open index.html with your browser of choice.

The folder Example contains two files. One of them, test_release.py, contains a script that shows how to use the mocaxextendpy library. This file can be used to test whether the installation of the software has been done properly. The file black_scholes_utils.py contains some functions imported in test_release.py.

The folder Library contains all that is related to the Library mocaxextendpy. Within the folder Library there are three folders: c_files, mocaxextendpy and shared_libs.

The folder c_files contains the files written in C that implement some of the routines used by mocaxextendpy. These routines can be called within mocaxextendpy through python bindings that rely on shared libraries. We have provided the bindings along with the shared libraries they rely on that have been written by us. The user can also generate these shared libraries using the files in c_files. Instructions of how to do this are given below along with how to obtain external shared libraries (cblas) the Python bindings rely on.

The folder mocaxextendpy contains the Python source code for the library.

Installation

First, Python should be installed on your computer. The mocaxextendpy package was developed in Python 3.7, so we recommend using this version, but previous Python versions should work too. A more detailed list of packages, along with their corresponding versions, that were part of the Python installation with which mocaxextendpy was developed, is given in requirements.txt, found within the Library folders for both Linux and Windows versions. Note that many of these packages are not directly needed to use mocaxextendpy.

It is sometimes convenient to work in a virtual environment. This, however, is not a requirement. To create a virtual environment you can use virtualenv. To install it run

pip install virtualenv

in your terminal (linux or windows). Choose a root directory for your project and run the following command

virtualenv env

This creates a new folder `env` in your project or root directory. Next activate it with the following command:

source env/bin/active

You can install all dependencies specified in requirements.txt with

pip install -r requirements.txt

Dependencies that must be installed

Other than the standard packages one normally uses with Python in numerical computing (such as numpy), the following are essential to use mocaxextendpy

Scikit-TT.

To install it simply enter:

pip install git+https://github.com/PGelss/scikit_tt

CBLAS.

The library mocaxextendpy relies on a series on a series of routines written in C. To run these routines, Python bindings have been generated. These binding rely on a series of shared libraries. Two of these (libtensorvals.so and libhommat.so, in the case of Linux) have been generated from the scripts in C provided in the c_files folder. The user may want to generate them instead of using the ones we provide. These shared libraries also rely on CBLAS.

To install CBLAS in linux:

Open terminal and run the following command:

sudo apt-get install libatlas-base-dev

This installs the cblas dependencies needed to both create .so files and to run the Python bindingds the package mocaxextendpy depends on.

To install in Windows:

For Windows we provide the shared library gslcblas.dll. This is needed to generate the shared libraries tensorvals.dll and hommat.dll (if user chooses to) and to run the library mocaxextendpy.

Generating shared libraries

We provide pre-built shared libraries that can be found in the folders shared_libs within Linux/Library or Windows/Library. These may or may not work in your system. These libraries should be placed in a folder that is within the environment variables of your Python installation.

Alternatively, the user may choose to generate the shared libraries using the C files provided in the folders c_files found in Linux/Library and Windows/Library. The description of how to generate the shared libraries that the wrappers in mocaxextendpy rely on is given next.

In Linux:

Create shared library for tensorVals.c by running in the folder c_files the following command:

gcc -fPIC -shared -o libtensorvals.so tensorVals.c -lcblas

This creates the shared library libtensorvals.so needed by the python wrapper found in rank_adaptive_C_interface.py. The shared library must be placed in a folder where Python can reach it. In Linux that is normally /usr/lib/ or if the Python installation is done through Anaconda, then it can also be in /anaconda/lib

Create shared library for homMat.c by running in the folder where homMat.c is:

gcc -fPIC -shared -o libhommat.so homMat.c -lcblas

This creates the shared library libhommat.so needed by the python wrapper found in rank_adaptive_C_interface.py. The shared library must be placed in a folder where Python can reach it. In Linux that is normally /usr/local/lib/ or if the Python installation is done through Anaconda, then it can also be in /anaconda/lib

In Windows:

Create shared library for tensorVals.c by running within the folder where tensorVals.C is:

gcc -shared -o tensorvals.dll tensorVals.c -I. -L. -lgslcblas

This creates the shared library tensorvals.dll needed by the python wrapper found in rank_adaptive_C_interface.py. The shared library must be placed in a folder that is part of the environment variables of your Python installation. For exampple, if your Python installation was done with Anaconda, one can place the shared libraries in Anaconda3\Library\bin

Create shared library for homMat.c by running within the folder where homMat.C is:

gcc -shared -o hommat.dll homMat.c -I. -L. -lgslcblas

This creates the shared library hommat.dll needed by the python wrapper found in rank_adaptive_C_interface.py. The shared library must be placed in a folder where Python can reach it. For example, if Python has been installed with Anaconda, one can place the shared libraries in Anaconda3\Library\bin

Installation of the mocaxextendpy package:

Once the Python wrappers have been generated and placed in a folder within Python's environment variables, the package mocaxextendpy must be in a folder within Python's system paths. For example, if Python has been installed with Anaconda, one can place the shared libraries in Anaconda3/lib/Python3.7/site-packages

Authors:

Mughda Kolte

Jayesh Chavan

Mariano Zeron

License:

mocaxextendpy - Python implementation of Tensor Extension Algorithms for Chebyshev Tensors

Copyright 2021 iRuiz Technologies, ltd.

See LICENSE.txt for details on the End User License Agreement.